



workout-guide 开源健身动作插画库

302 个动作 · 906 张 SVG 矢量插画 · 一个 TypeScript npm 包 —— 做健身 App 的“图标库”，但是给动作作用的

GitHub 1.3k★ · 曾登周榜第 5

@bryllim/workout-guide

代码 MIT / 插画 CC BY-SA 4.0

拾遗任务 4e83f48a · 来源：抖音 JavaPub 《做健身 App 必备！302 个动作全部开源》 · 仓库

github.com/bryllim/workout-guide · 2026-10-03

0 一句话 & 疑问地图（疑问即目录）



它是什么？

workout-guide 是一个“动作插画素材库 + 查询工具包”：302 个健身动作，每个动作 3 帧（起-中-止）统一风格的 SVG 线稿插画，打包成框架无关的 TypeScript npm 包，再配一个可搜索的在线画廊。做健身 App 最贵的不是代码，是**几百张风格统一的动作图**——这个项目把这块成本直接归零。

Q1 为什么需要它？没有它之前人们怎么做？

健身 App（Keep、workout-cool、Hevy 这类）每个训练动作都要配图。自己做的三条老路：

老路	代价
请插画师画 300+ 张动作图	数千到上万元，周期以月计，风格难统一
扒 Everkinetic 等老数据集的 PNG	风格陈旧不统一、无结构化元数据、License 模糊
用照片/视频	版权风险大、体积大、深浅主题难适配

workout-guide 的答案：开源 + 统一矢量风格 + 结构化元数据 + 明确双许可，一次解决四个痛点。

Q2 302 个动作哪来的？和 Everkinetic 什么关系？

作者 Bryl Lim 在 [Everkinetic](#)（一个老牌开源动作数据集，CC BY-SA 4.0）基础上二次创作：把老图重绘成统一风格线稿、补画大量新动作、补齐中间帧。实测 manifest：906 帧中 **76 帧带 Everkinetic 上游溯源标注**（记录来源 URL + 改动说明），其余为作者新绘。这是开源二创的**教科书操作：逐帧写清“从哪来、改了什么”**，CC BY-SA 的署名-相同方式共享义务落到了每个文件。



图2-1 二创链：继承 CC BY-SA 4.0，逐帧 attribution 记来源与改动

Q3 仓库怎么组织？核心只有 80 行代码？

npm workspaces monorepo，四块：

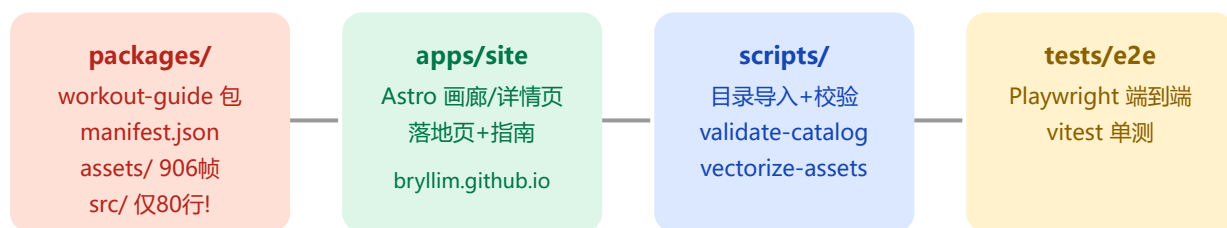


图3-1 monorepo 四块结构

API 面极小：**4 个函数 + 2 个 Map 索引，共 80 行 TypeScript**。价值大头在资产和元数据，代码只是“查询壳”——**内容型开源项目的典型形态**。

Q4 manifest.json 数据结构长什么样？

一个 302 元素的数组，每条动作记录：

```
{ "id": "exercise-bench-press", "slug": "bench-press", "name": "Bench Press",
  "exerciseType": "weight_reps", // 5 种：
  weight_reps/bodyweight_reps/duration/distance_duration/assisted_bodyweight
  "equipment": "Barbell", // 17 种器械
  "primaryMuscle": "Chest", // 20 个主肌群；secondaryMuscles 还有协同肌
  "isStretch": false,
  "frames": [ { "index": 1, "path": "assets/bench-press/frame-1.svg", "width": 512, "height": 512,
    "attribution": { creator, license: "CC BY-SA 4.0",
      source: { name:"Everkinetic", url, changes:"Rasterized...vector-
        traced" } } }, ...共3帧 ] }
```

实测分布：器械 **Bodyweight 111 / Dumbbell 45 / Machine 35 / Barbell 29**；肌群 **Core 45 / Glutes 38 / Quads 35 / Chest 26**。每帧自带宽高和出处——数据即合同，App 端不用猜。

Q5 为什么每动作只有 3 帧？3 帧够吗？

动作示范的目的是 **看清轨迹，不是以假乱真**。起势→中程→止点 3 帧恰好覆盖：起点教摆位、中程教轨迹、止点教收缩。对比方案：视频几十 MB、GIF 数 MB、3 帧 SVG 总共约 **7KB/动作**。循环播放时 1→2→3→2→1 的“钟摆式”循环就自然连贯（本指南实验 4 的图鉴就是这么做的，CSS 十几行搞定）。



循环播放 1→2→3→2→1（钟摆式）= 一个完整动作示范

图5-1 三帧法：最小帧数覆盖完整动作语义

Q6 双许可（MIT + CC BY-SA 4.0）怎么理解？商用要注意什么？

一个仓库两套许可，按“内容类型”划界：

部分	许可	你能干什么	义务
代码 / 文档 (src、scripts、site)	MIT	随使用、改、闭源商用	保留版权声明
插画资产 (assets/ 全部 906 帧)	CC BY-SA 4.0	免费商用	署名 (标注 Bryl Lim / workout-guide) + 相同方式共享 (你若修改分发这些图，修改版也得 CC BY-SA 4.0)

⚠ 实操要点：App 里用原版图 = 署名即可（放“关于”页）；**改了图再分发，改的部分也必须开源同许可**。npm 包的 license 字段特意写成“SEE LICENSE IN LICENSES.md”逼使用户看两份许可。溯源链上游 Everkinetic 也是 CC BY-SA 4.0，许可兼容不冲突。

Q7 图片怎么到用户手机上？CDN 分发链路是什么？

getAssetUrl('push-up', 1) 默认返回：

```
https://cdn.jsdelivr.net/npm/@bryllim/workout-guide@1.0.0/assets/push-up/frame-1.png
```

npm 包本身就是"CDN 源"：jsDelivr 自动镜像 npm 上的任何包并提供全球边缘缓存——不花一分钱基础设施，App 换 URL 版本号即灰度升级 (@1.0.0→@latest)。也可以 {baseUrl:'https://myapp.com/assets/'} 指向自托管。三个真实坑（实验全部踩到）：

坑1：发布版 manifest 帧路径是 **.png** 不是 **.svg**——仓库里 svg+png 双份（svg 2.3MB 是矢量源，png 兼容兜底），npm 包只发 manifest 引用的 PNG；坑2：{baseUrl} 会拼出 **assets/assets/** 双前缀——frame.path 自带 assets/ 前缀，baseUrl 应指向包根而不是 assets 目录；坑3：**SVG 是白色线稿+透明底**，放在白色背景上直接"隐身"——官方画廊是深色主题，自建 UI 必须给深色容器（或改 fill）。

Q8 "框架无关的 npm 包"是怎么做到的？

三招：① **预构建 dist**（发布时带好 index.js ESM + index.cjs CJS + index.d.ts 类型，装了就能用，不需要用户配构建器）；② **package.json exports 双入口**（. 主入口、./manifest.json、./assets/* 直取原始资产）；③ **零运行时依赖**——不绑 React/Vue，输出纯数据对象。React 直接 getAssetUrl 放 ，React Native 用 require() 静态图，Vue/原生 JS 同理。实测 npm install 后 Node ESM 直接 import 成功（实验 3）。



图8-1 一包多端：预构建 + exports + 零依赖 = 框架无关

Q9 搜索为什么大小写、重音、符号都打不倒它？

秘密在 normalizeSearchText 的归一化管道，实测 "Café & Lunge!" → "cafe and lunge"：

```
toLowerCase()           // 大小写统一
.normalize('NFD') + 去组合重音符 // café → cafe（把 é 拆成 e + ´再删掉）
.replace(/&/g, ' and ') // 符号转词：& 就是 and
.replace(/[^a-z0-9]+/g, ' ') // 其余符号→空格
.trim() + 空格折叠       // 多空格合一
```

查询和被查字段走 **同一个归一化函数再比较**——“规范化后比相等/包含”，**比上搜索引擎轻一个数量级，302 条本地内存过滤瞬间完成**。多词查询用“每个 token 都须命中”（AND 语义）。

Q10 906 张图 + manifest 怎么保证不出现坏数据？

仓库 scripts/ 里有 validate-catalog.mjs 校验脚本 + npm run check 总门，理念就是知识库学过的**质检 Gate 与自我纠错循环**：目录导入→校验→过门才能合并。我照着这个思路自己写了零依赖质检脚本（实验 1），7 项检查全过：

- ✓ 302 动作 = manifest 记录数；
- ✓ 906 帧记录 = 906 个磁盘 SVG（账实相符）；
- ✓ 每帧 512×512；
- ✓ 文件头含 <svg>；
- ✓ 每帧 attribution 带 CC BY-SA 4.0；
- ✓ 每动作恰好 3 帧；
- ✓ slug 无重复。→ 质检 0 异常

数据集项目的可信度不靠宣传靠对账：声明数、文件数、字段值三方一致才算数。

Q11 它和同类项目什么关系？什么时候选它？

项目	形态	选它的场景
workout-guide	插画库 + 查询包（302 动作）	要风格统一的活动插画、要自建 UI、要轻量
yuhonas/free-exercise-db	纯 JSON 数据集（800+ 动作）+ 英文说明	要文字说明/属性，图不讲究
GitHub topic: workout 数据集	1300+ 动作 GIF 数据	要海量但接受 GIF 体积与风格混杂
Snouzy/workout-cool	完整健身平台（产品）	要现成 App 而非素材

一句话：要“图”选 workout-guide，要“字”选 free-exercise-db，要“现成产品”选 workout-cool。

Q12 我怎么用它做一个健身 App 的最小动作页？

四步（全部真实跑通，见下一节）：npm 装包 → searchExercises 按肌群/器械筛动作 → getAssetUrl 拿图（或本地 assets）→ 三帧 CSS 动画轮播。实验 4 产出的 exercise_gallery.html 就是零框架的最小完整示例：302 卡片 + 筛选 + 悬停播放，共 139KB。

动手实验（全部真实运行，输出为实测）

实验 1+2 | manifest 数据体检 + 资产完整性质检 Gate（零依赖，Python 标准库）

```
# 怎么操作: python 01_manifest_audit.py (读 repo 的 manifest + assets 对账)
[1] manifest 动作数: 302 | 帧记录: 906 (302x3=906 PASS) | 磁盘 SVG: 906 (PASS)
[3] 器械 Top: Bodyweight 111 / Dumbbell 45 / Machine 35 / Barbell 29 (共17种)
    肌群 Top: Core 45 / Glutes 38 / Quads 35 / Chest 26 (共20群)
[4] 质检 Gate: 路径存在/512尺寸/SVG头/CC溯源/3帧/slug唯一 → 0 异常 PASS
[5] Everkinetic 溯源帧: 76/906 (其余为作者新绘)
```

✓ 预期结果（实测）：学会了“数据集对账”方法——不数宣传页的数，直接对 manifest 和磁盘

实验 3 | 真装 npm 包, 调四个 API (Node 24 ESM)

```
# npm install @bryllim/workout-guide (35MB, 含 dist+assets+manifest)
getExercise('push-up') → Push-up | Bodyweight | Chest | 3帧
searchExercises('press',{equipment:'Dumbbell'}) → 6 个 (Arnold Press...)
searchExercises('',{primaryMuscle:'chest',equipment:'bodyweight'}) → 12 个
getAssetUrl('push-up',1) → https://cdn.jsdelivr.net/npm/@bryllim/workout-guide@1.0.0/assets/push-up/frame-1.png
normalizeSearchText('Café & Lunge!') → "cafe and lunge"
getExercise('不存在') → null (不抛异常, 查询API契约友好)
```

✅ 预期结果 (实测) : 装包 4 秒, 零配置 import 成功; 同时踩到 3 个真实坑 (.png 路径 / assets 双前缀 / 白线稿需深底)

实验 4 | 生成"302 动作图鉴浏览器" (零依赖静态页 + CSS 三帧动画)

```
# python 03_build_gallery.py → exercise_gallery.html (139KB) + assets_svg/ 906 帧 (2.3MB)
功能: 302 卡片 · 20 肌群/17 器械下拉筛选 · 悬停播放 起-中-止 steps 动画
验证: Edge 无头截图 → 302 卡全部渲染 (首测白底"隐身", 加深色底后修复)
```

✅ 预期结果 (实测) : 一个 HTML + 一个资产目录 = 可离线浏览的完整动作库, 双击即用

📁 这个项目给我什么 (能力清单)

- **直接拿来用**: 自研健身 App/小程序时 302 动作插画 + 元数据即插即用 (署名即可免费商用)
- **动作数据骨架**: 器械/肌群/动作类型/计次方式的结构化字段, 直接当自己数据库的种子数据
- **三帧动画模式**: 几 KB 的 SVG 实现动作示范, 比视频/GIF 轻百倍, 适配深浅主题
- **数据集工程范本**: manifest 即合同 + 逐条溯源 + 校验脚本, 可复用到任何素材库项目
- **内容型开源范本**: 代码 80 行、价值在内容——"做开源不一定要写复杂代码"
- **jsDelivr 分发术**: npm 包当 CDN 源, 零基础设施全球分发



一页记忆卡

```
workout-guide = 302 动作 × 3 帧 SVG + manifest 元数据 + 80 行查询 API
许可   = 代码 MIT (随使用) / 插画 CC BY-SA 4.0 (署名+改图同许可)
分发   = npm 包即 CDN 源 (jsDelivr 自动镜像), 版本号即灰度
三个坑 = npm 发布 .png 非 .svg; baseUrl 双 assets 前缀; 白线稿必须深色底
一句话 = 做健身 App, 图的部分已经有人替你画好了
```